

Universidad del Valle de Guatemala

Facultad de Ingeniería

CC3067 Redes, sección 10

Laboratorio 5

WordPress en dos capas sobre VirtualBox

Nicolás Concuá, 23197

Daniela Ramírez, 23053

Fernando Mendoza, 19644

Guatemala, 10 de noviembre de 2026

Índice

1. Introducción 3
2. Entorno y versiones 3
3. Topología y direccionamiento 4
4. Implementación 6
    4.1 Máquinas virtuales 6
    4.2 Acceso por SSH 6
    4.3 Firewall 7
    4.4 Base de datos 7
    4.5 Servidor web y WordPress 7
    4.6 Permisos 8
    4.7 SFTP y FileZilla 8
    4.8 HTTPS 10
    4.9 Resolución local 11
5. Cliente y servidor 12
6. Pruebas 12
7. Problemas encontrados 13
8. Conclusiones 13
Referencias 13
Anexo. Reconstrucción del laboratorio 14

1. Introducción

En este laboratorio montamos un sitio WordPress en dos máquinas virtuales: una para el servidor web y otra para la base de datos. Las dos corren Ubuntu Server 26.04.1 LTS sobre VirtualBox y se administran solo por SSH. El sitio se publica por HTTPS con un certificado de Let's Encrypt y se abre desde el cliente con el nombre `redes-lab5-wp.duckdns.org`, que se resuelve con el archivo hosts.

La configuración está en scripts dentro del repositorio del grupo (github.com/nicoCT04/Lab5-Redes). Todos leen los mismos parámetros de `config/lab5.env`, y las contraseñas están en `secretos.env`, que no se sube al repositorio. Con esos scripts el laboratorio se puede reconstruir desde cero siguiendo los pasos de la sección 4.

2. Entorno y versiones

Las máquinas virtuales corren en VirtualBox 7.2.0, un hipervisor de tipo 2 que se ejecuta como aplicación sobre Windows 11. Es uno de los permitidos por el enunciado y permite asignar a cada VM un adaptador NAT y uno host-only, que es lo que necesita el diseño.

Las VM se instalaron con el instalador de Ubuntu Server en modo *autoinstall*: una ISO adicional con el archivo `user-data` define el nombre, el usuario, la llave SSH y la red, y la instalación termina sin intervención.

Tabla 1. Versiones utilizadas.

Componente	Versión
Anfitrión	Windows 11, VirtualBox 7.2.0
Máquinas virtuales	Ubuntu Server 26.04.1 LTS, 2 vCPU, 2 GB de RAM, 15 GB de disco
SSH y SFTP	OpenSSH 10.2p1
Firewall	UFW 0.36.2
Servidor web	Nginx 1.28.3, PHP-FPM 8.5.4
CMS	WordPress 7.1.4, WP-CLI 2.12.0
Base de datos	MySQL 8.4.11 LTS
Certificados	certbot 4.0.0
Cliente	Windows 11, FileZilla, navegador

Los paquetes de las VM vienen del repositorio oficial de Ubuntu 26.04, que mantiene actualizaciones de seguridad durante todo el soporte de la versión. MySQL 8.4 es la versión LTS que trae la distribución. Usamos Nginx con PHP-FPM en lugar de Apache porque consume menos memoria, algo que pesa con 2 GB por VM.

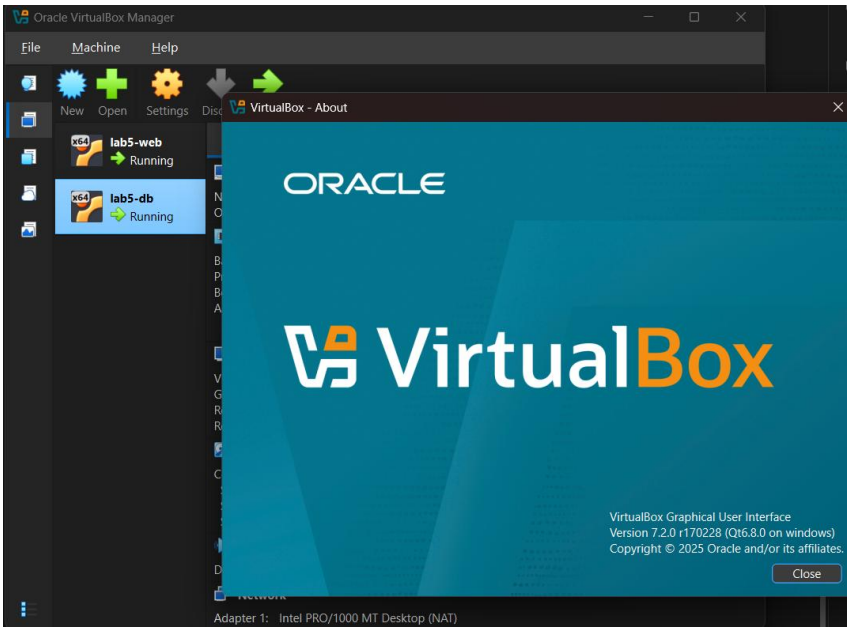


Figura 1. VirtualBox 7.2.0 con lab5-web y lab5-db en ejecución.

3. Topología y direccionamiento

Cada VM tiene dos adaptadores. El adaptador 2 está en la red host-only 192.168.56.0/24, sin DHCP; ahí se ofrecen todos los servicios y las direcciones son fijas. El adaptador 1 usa NAT y solo sirve para que la VM salga a Internet (paquetes, Let's Encrypt y la API de DuckDNS). Desde fuera no se puede iniciar ninguna conexión hacia las VM por ese adaptador.

El host tiene la dirección 192.168.56.1 en el adaptador host-only y desde ahí usa SSH, FileZilla y el navegador.

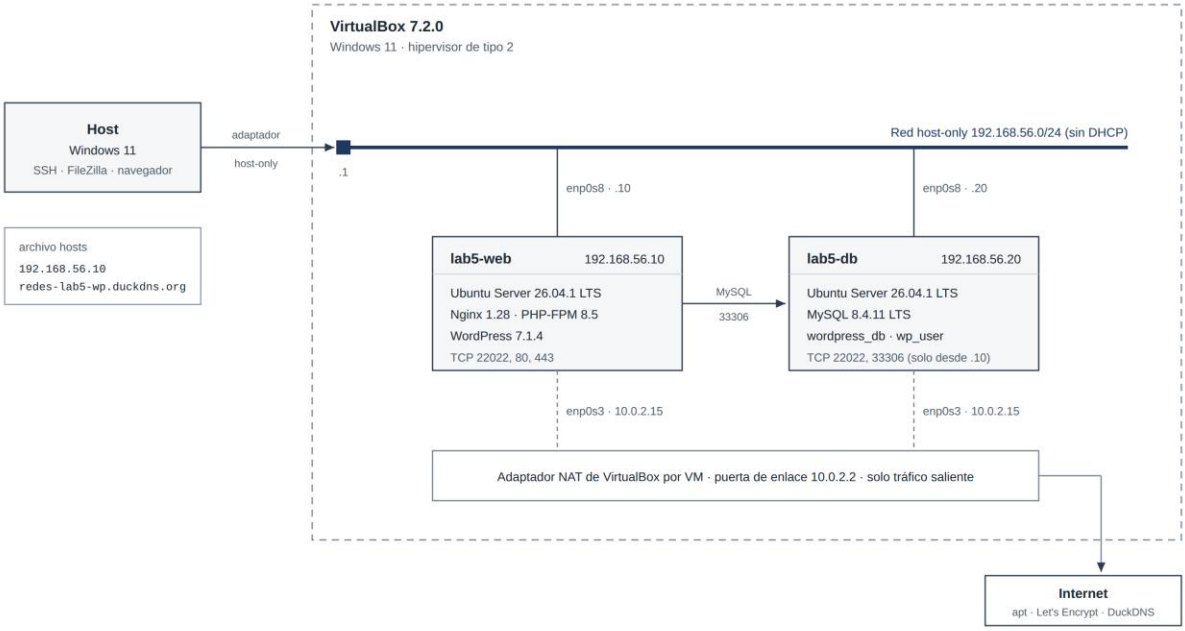


Figura 2. Topología del laboratorio.

Tabla 2. Direcciones y puertos.

Equipo	Dirección	Puertos abiertos
Host (host-only)	192.168.56.1	ninguno
lab5-web	192.168.56.10	22022 (SSH/SFTP), 80, 443
lab5-db	192.168.56.20	22022 (SSH), 33306 (MySQL, solo desde .10)
redes-lab5-wp.duckdns.org	192.168.56.10	resuelto por el archivo hosts

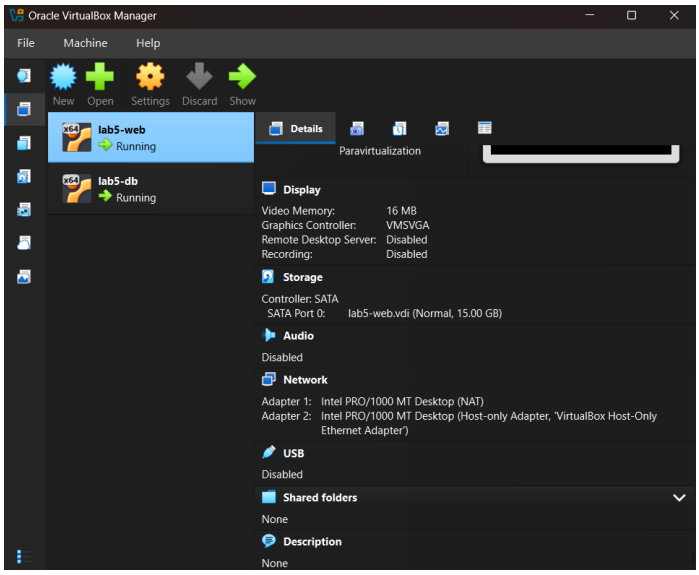


Figura 3. Adaptadores de lab5-web en VirtualBox.

4. Implementación

Los scripts de las VM se ejecutan desde el cliente con `ssh web "cd ~/lab5 && sudo bash scripts/..."`. El orden importa: primero SSH y firewall, después la base de datos, porque la instalación de WordPress necesita conectarse a ella, y al final la capa web y HTTPS.

4.1 Máquinas virtuales

`infra-windows/crear-vms.sh` crea las dos VM con `VBoxManage` (2 vCPU, 2 GB de RAM y 15 GB de disco), las conecta a NAT y a la red host-only y arranca el instalador con la ISO de *autoinstall*. Esa ISO define el nombre del equipo, el usuario `sysadmin` con la llave pública del grupo, SSH sin contraseña y las direcciones fijas. `infra-windows/subir.sh` copia después los scripts a `~/lab5` en cada VM. La consola de VirtualBox solo se usó durante la instalación.

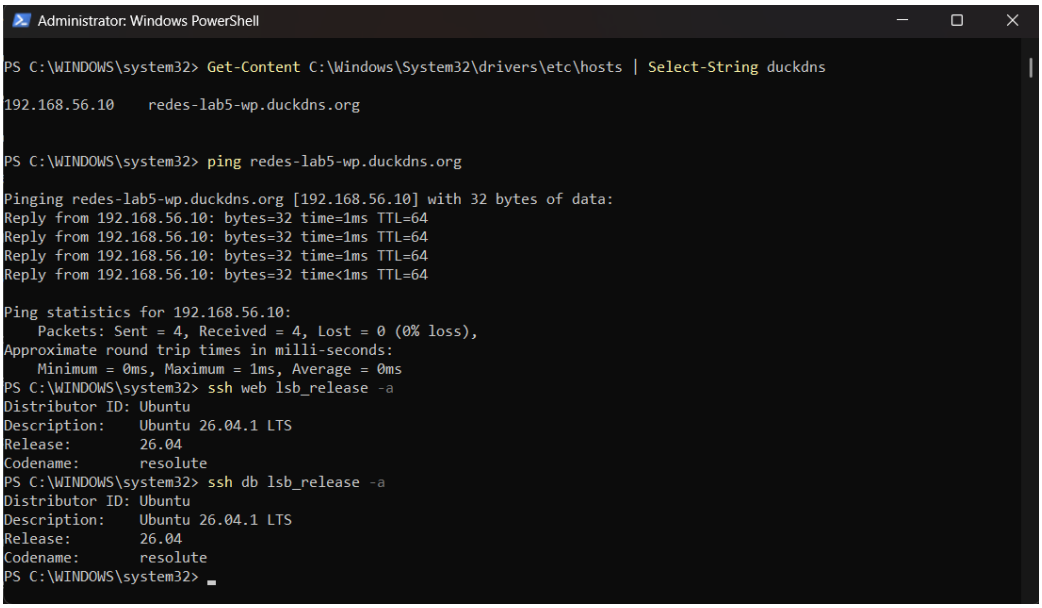


Figura 4. Archivo hosts, ping al dominio y versión del sistema en ambas VM.

4.2 Acceso por SSH

`scripts/comun/01-ssh-puerto.sh` mueve SSH al puerto 22022 y deshabilita el acceso como root y la autenticación por contraseña. La configuración va en `/etc/ssh/sshd_config.d/10-lab5.conf` y se valida con `sshd -t` antes de reiniciar, para no perder el acceso. En Ubuntu 26.04 el puerto lo abre `ssh.socket`, así que el script también recarga systemd y reinicia el socket.

El cambio de puerto no impide el acceso a nadie; eso lo hacen la llave y el firewall. Lo que sí reduce son los intentos automáticos que buscan el puerto 22.

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/comun/01-ssh-puerto.sh"
LISTEN 0  4096   0.0.0.0:22022   0.0.0.0:*   users:(("sshd",pid=13511,fd=3),("systemd",pid=1,fd=82))
OK: SSH escuchando en 22022
PS> Test-NetConnection 192.168.56.10 -Port 22
TcpTestSucceeded : False
```

Figura 5. SSH por el puerto 22022. El puerto 22 ya no responde.

4.3 Firewall

`scripts/comun/02-ufw.sh` deja UFW con la política de denegar todo lo entrante y abre solo lo necesario. En lab5-web: 22022 desde la red del laboratorio, 80 (que solo redirige a HTTPS) y 443. En lab5-db: 22022 desde la red del laboratorio y 33306 únicamente desde 192.168.56.10, de modo que el cliente no puede conectarse a MySQL aunque esté en la misma red.

```
$ ssh web "sudo ufw status verbose"
Default: deny (incoming), allow (outgoing), disabled (routed)
22022/tcp                  ALLOW IN    192.168.56.0/24            # SSH/SFTP lab5
80/tcp                     ALLOW IN    Anywhere                   # HTTP (redirige a HTTPS)
443/tcp                    ALLOW IN    Anywhere                   # HTTPS WordPress
$ ssh db "sudo ufw status verbose"
Default: deny (incoming), allow (outgoing), disabled (routed)
22022/tcp                  ALLOW IN    192.168.56.0/24            # SSH/SFTP lab5
33306/tcp                  ALLOW IN    192.168.56.10              # MySQL desde web
```

Figura 6. Reglas de UFW en lab5-web y lab5-db.

4.4 Base de datos

`scripts/bd/01-mysql.sh` instala MySQL 8.4 y agrega `/etc/mysql/mysql.conf.d/zz-lab5.cnf` con `bind-address = 192.168.56.20`, `port = 33306` y `mysqlx = 0`. El nombre empieza con zz porque MySQL lee esos archivos en orden alfabético y se queda con el último valor; con otro nombre, `mysqld.cnf` volvía a dejar el servicio en 127.0.0.1.

La base `wordpress_db` y el usuario `wp_user` son exclusivos de WordPress. `wp_user` solo puede conectarse desde 192.168.56.10 y solo tiene privilegios sobre esa base. El usuario root de MySQL usa `auth_socket`: únicamente entra con `sudo mysql` desde la propia VM, que es como administramos la base por SSH.

```
$ ssh -t db "sudo mysql -e 'SELECT user, host, plugin FROM mysql.user;'"
wp_user          192.168.56.10   caching_sha2_password
root             localhost       auth_socket
$ ssh -t db "cd ~/lab5 && sudo bash scripts/bd/01-mysql.sh"
GRANT ALL PRIVILEGES ON `wordpress_db`.* TO `wp_user`@`192.168.56.10`
mysql  Ver 8.4.11-0ubuntu0.26.04.1 for Linux on x86_64 ((Ubuntu))
LISTEN 0      151    192.168.56.20:33306      0.0.0.0:*    users:(("mysqld",pid=2609,fd=19))
```

Figura 7. Administración de MySQL por SSH: usuarios, privilegios y puerto.

4.5 Servidor web y WordPress

`scripts/web/01-stack-web.sh` instala Nginx, PHP-FPM con las extensiones de WordPress, el cliente `mysql` y WP-CLI. `scripts/web/02-wordpress-permisos.sh` primero prueba la conexión a la base remota con `wp_user` y luego instala WordPress con WP-CLI, con la base en 192.168.56.20:33306. En `wp-config.php` activamos `DISALLOW_FILE_MODS` y `DISALLOW_FILE_EDIT`, porque el servidor web no tiene permiso de escritura (sección 4.6), y `FORCE_SSL_ADMIN` para que el panel solo funcione por HTTPS.

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/pruebas/pruebas-servidor.sh web"
3 packets transmitted, 3 received, 0% packet loss, time 2007ms
Connection to 192.168.56.20 33306 port [tcp/*] succeeded!
nc: connect to 192.168.56.20 port 3306 (tcp) timed out: Operation now in progress
CURRENT_USER()          @@hostname   @@port
wp_user@192.168.56.10   lab5-db      33306
```

Figura 8. Desde lab5-web: ping a la base, puertos 33306 y 3306, y login de wp_user.

4.6 Permisos

El sitio pertenece a `wpadmin`, el único usuario que puede escribir en él. El grupo es `www-data`, con el que corren Nginx y PHP-FPM, que así pueden leer los archivos pero no modificarlos. Los demás usuarios, incluido `sysadmin`, no tienen acceso. Si alguien aprovechara una falla de WordPress, su código correría como `www-data` y no podría cambiar el sitio.

Tabla 3. Permisos del sitio.

Ruta	Dueño:grupo	Modo
/var/www	root:root	755
/var/www/wordpress y subdirectorios	wpadmin:www-data	2750
Archivos del sitio	wpadmin:www-data	640

El bit setgid (el 2 de 2750) hace que lo que sube `wpadmin` herede el grupo `www-data`. `/var/www` pertenece a root porque así lo exige el chroot de SFTP.

```
chown -R wpadmin:www-data /var/www/wordpress
find /var/www/wordpress -type d -exec chmod 2750 {} +
find /var/www/wordpress -type f -exec chmod 640 {} +
```

```
drwxr-xr-x 3 root    root     4096 Nov  9 14:01 /var/www
drwxr-s--- 5 wpadmin www-data 4096 Nov  9 14:01 /var/www/wordpress
-rw-r-----  1 wpadmin www-data   405 Nov  9 14:01 index.php
touch: cannot touch '/var/www/wordpress/prueba': Permission denied        (www-data)
touch: setting times of '/var/www/wordpress/prueba': Permission denied    (sysadmin)
-rw-r--r-- 1 wpadmin www-data 0 Nov  9 14:04 /var/www/wordpress/prueba    (wpadmin)
```

Figura 9. Permisos resultantes y prueba de escritura con www-data, sysadmin y wpadmin.

4.7 SFTP y FileZilla

`scripts/web/03-sftp.sh` agrega a sshd un bloque `Match User wpadmin` con `ChrootDirectory /var/www`, `ForceCommand internal-sftp -u 0027` y autenticación por contraseña solo para ese usuario. `wpadmin` no puede abrir una terminal, solo ve la carpeta `wordpress` y lo que sube queda con permisos 640 o 750.

En FileZilla nos conectamos a `sftp://192.168.56.10` en el puerto 22022 como `wpadmin`, subimos un archivo HTML a `/wordpress` y lo abrimos en el navegador.

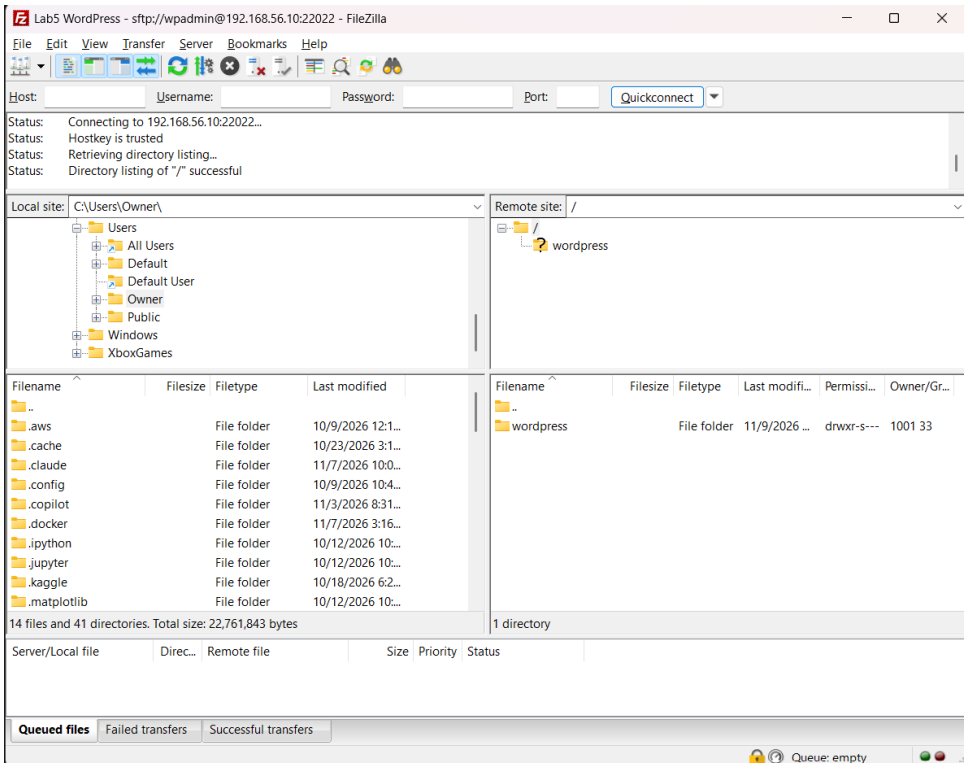


Figura 10. FileZilla conectado como wpadmin. Solo se ve /wordpress.

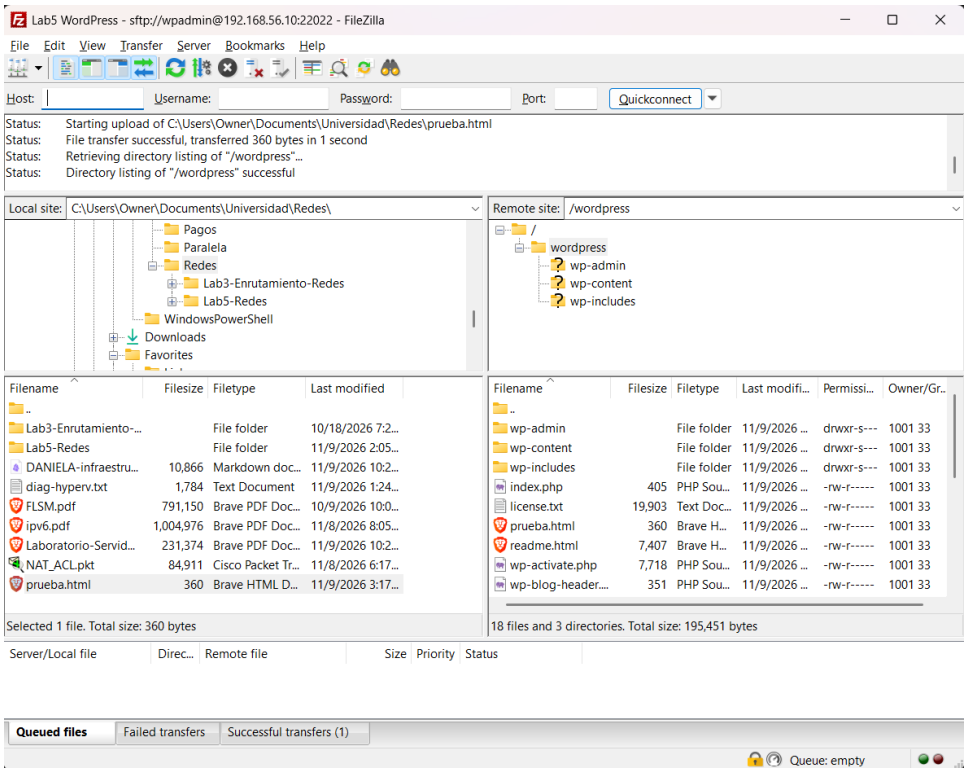


Figura 11. Subida de prueba.html.



Figura 12. El archivo subido, servido por HTTPS.

4.8 HTTPS

Let's Encrypt solo emite un certificado si se demuestra control sobre el dominio. Los retos HTTP-01 y TLS-ALPN-01 requieren que sus servidores se conecten a la VM por el puerto 80 o 443, y nuestras VM no son accesibles desde Internet. Usamos el reto DNS-01, que solo pide publicar un registro TXT en el DNS público. Por eso el dominio es un subdominio de DuckDNS, que permite crear ese registro por API; un dominio inventado como `.local` no se puede validar.

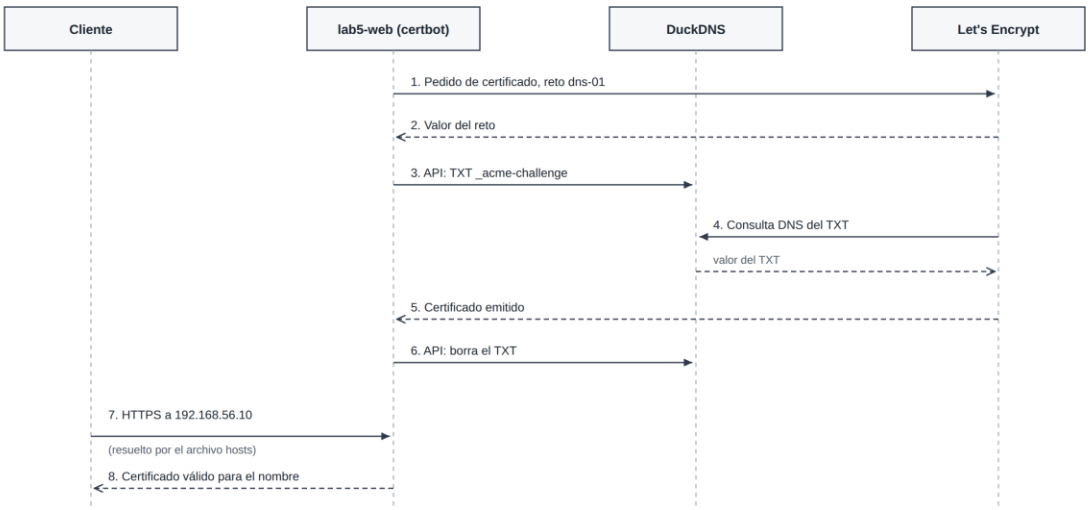


Figura 13. Emisión del certificado con el reto DNS-01.

`scripts/web/04-https.sh` instala certbot con dos scripts que publican y borran el TXT en DuckDNS. Antes de pedir el certificado real lo probamos contra el entorno de pruebas de Let's Encrypt, que no cuenta para el límite de emisiones. Nginx redirige el puerto 80 a HTTPS, acepta TLS 1.2 y 1.3, envía HSTS y bloquea `wp-config.php` y `xmlrpc.php`. `certbot.timer` se encarga de la renovación.

El nombre existe en el DNS público solo para la validación. En el cliente, el archivo hosts lo dirige a 192.168.56.10, y el navegador acepta el certificado porque coinciden el nombre y la cadena de confianza; la dirección IP no interviene.

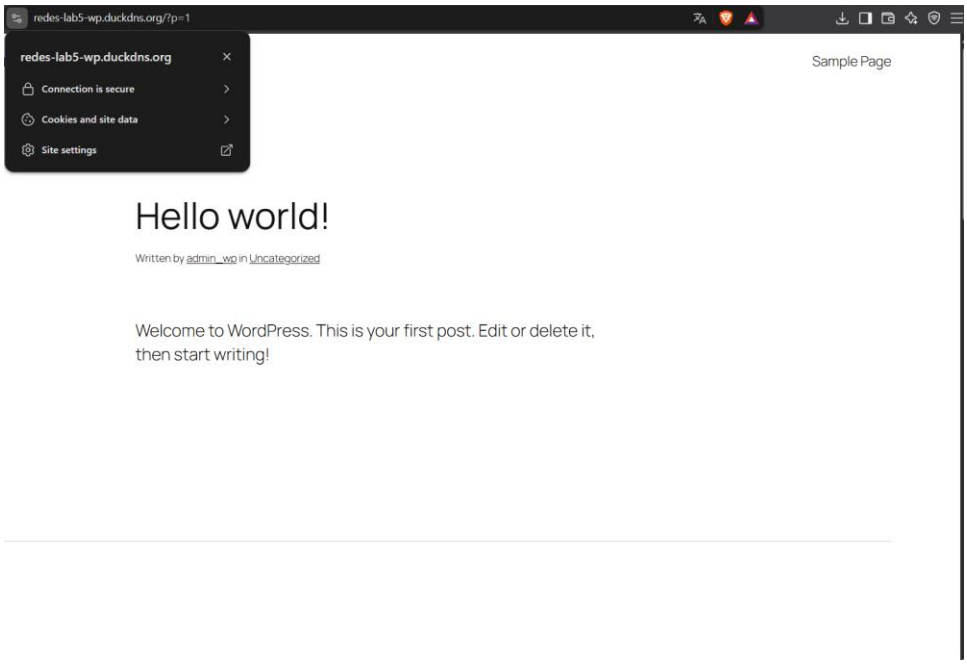


Figura 14. WordPress por HTTPS en el navegador del host.

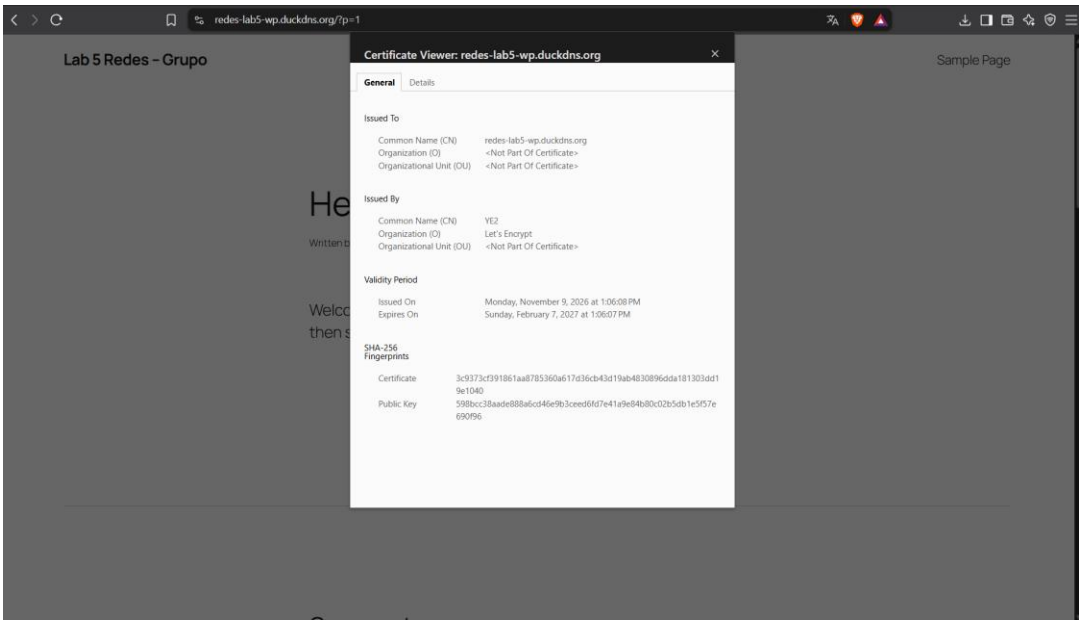


Figura 15. Certificado de Let's Encrypt para redes-lab5-wp.duckdns.org.

4.9 Resolución local

El archivo `C:\Windows\System32\drivers\etc\hosts` del cliente tiene la línea `192.168.56.10 redes-lab5-wp.duckdns.org`. Windows consulta ese archivo antes que el DNS, así que el nombre resuelve a la VM web aunque en DuckDNS apunte a otra dirección (figura 4).

5. Cliente y servidor

Una misma máquina puede ser servidor en una conexión y cliente en otra. lab5-web es el servidor que atiende al navegador, pero frente a la base de datos es el cliente: PHP y el cliente `mysql` se conectan a `mysqld` en lab5-db.

Tabla 4. Clientes y servidores en cada conexión.

Conexión	Cliente (aplicación, equipo)	Servidor (aplicación, equipo)
Sitio web	Navegador, host	Nginx y PHP-FPM, lab5-web
Archivos	FileZilla, host	OpenSSH (SFTP), lab5-web
Administración	ssh, host	sshd, lab5-web y lab5-db
Datos	PHP y mysql, lab5-web	mysqld, lab5-db

6. Pruebas

Las pruebas están en `scripts/pruebas`: `pruebas-servidor.sh` corre dentro de cada VM y `pruebas-host-windows.ps1` desde el host.

Tabla 5. Resultados de las pruebas.

Prueba	Resultado
ping entre lab5-web y lab5-db	0 % de pérdida
lab5-web hacia 192.168.56.20:33306	conecta
lab5-web hacia 192.168.56.20:3306	sin respuesta
Login de wp_user desde lab5-web	wp_user@192.168.56.10, 12 tablas de WordPress
Cliente hacia 22022 y 443	abiertos
Cliente hacia 22 y 33306	sin respuesta
Escritura en el sitio	solo wpadmin; www-data y sysadmin reciben Permission denied
SFTP con FileZilla	archivo subido y visible por HTTPS
HTTPS	200, certificado de Let's Encrypt válido
HTTP	301 hacia HTTPS

```
PS> powershell -ExecutionPolicy Bypass -File scripts\pruebas\pruebas-host-windows.ps1 -Dominio redes-
lab5-wp.duckdns.org
192.168.56.10  RemotePort 22022  TcpTestSucceeded : True
192.168.56.10  RemotePort 443    TcpTestSucceeded : True
192.168.56.20  RemotePort 22022  TcpTestSucceeded : True
192.168.56.10  RemotePort 22     TcpTestSucceeded : False
192.168.56.20  RemotePort 33306  TcpTestSucceeded : False
lab5-web   Description: Ubuntu 26.04.1 LTS
lab5-db    Description: Ubuntu 26.04.1 LTS
HTTP/1.1 200 OK
```

Figura 16. Pruebas desde el host Windows.

7. Problemas encontrados

Tabla 6. Problemas y soluciones.

Problema	Solución
apt no resolvía nombres; las VM recibían los DNS de la red de la universidad.	DNS fijos (1.1.1.1 y 8.8.8.8) en netplan para la interfaz NAT.
MySQL escuchaba en 127.0.0.1.	Renombrar la configuración a zz-lab5.cnf.
El puerto de SSH no cambiaba al reiniciar el servicio.	Reiniciar ssh.socket.
Las VM se congelaban en la laptop con Windows por Hyper-V.	Desactivar Hyper-V, integridad de memoria y VBS.
El instalador pedía confirmar el autoinstall.	Responder con VBoxManage keyboardputstring.
La traducción de WordPress 7.1.4 no existía al instalar.	Instalar en inglés y activar el español después.

8. Conclusiones

Separar el servidor web y la base de datos solo sirve si la red lo respeta. Con `bind-address`, la regla de UFW y un usuario limitado a su base, MySQL no es accesible ni desde el cliente.

Con los permisos queda claro quién hace qué: `wpadmin` publica, `www-data` sirve y nadie más toca los archivos.

El reto DNS-01 nos permitió tener un certificado público para un servidor privado, y los scripts hicieron que la instalación fuera la misma en el prototipo y en la laptop de la demostración.

Referencias

[1] Oracle. VirtualBox User Guide, cap. 6: Virtual Networking. https://www.virtualbox.org/manual/

[2] Canonical. Ubuntu Server documentation: OpenSSH, UFW, Netplan. https://documentation.ubuntu.com/server/

[3] Oracle. MySQL 8.4 Reference Manual: Using Option Files. https://dev.mysql.com/doc/refman/8.4/en/

[4] OpenBSD. sshd_config(5). https://man.openbsd.org/sshd_config

[5] NGINX. ngx_http_ssl_module. https://nginx.org/en/docs/

[6] Let's Encrypt. Challenge Types; Rate Limits. https://letsencrypt.org/docs/

[7] EFF. Certbot User Guide. https://eff-certbot.readthedocs.io/

[8] DuckDNS. Spec. https://www.duckdns.org/spec.jsp

[9] WordPress.org. Hardening WordPress; WP-CLI. https://developer.wordpress.org/

Anexo. Reconstrucción del laboratorio

```
# En el host (Git Bash)
infra-windows/crear-vms.sh
infra-windows/crear-vms.sh --terminar
infra-windows/subir.sh web 22
infra-windows/subir.sh db 22

# Luego, en orden, desde PowerShell
ssh -t web "cd ~/lab5 && sudo bash scripts/comun/01-ssh-puerto.sh"   # y lo mismo en db
ssh -t web "cd ~/lab5 && sudo bash scripts/comun/02-ufw.sh web"
ssh -t db  "cd ~/lab5 && sudo bash scripts/comun/02-ufw.sh db"
ssh -t db  "cd ~/lab5 && sudo bash scripts/bd/01-mysql.sh"
ssh -t web "cd ~/lab5 && sudo bash scripts/web/01-stack-web.sh"      # 02, 03 y 04 igual
```